

Multilingual Visual Question Answering for Visually Impaired People

```
import os
import sys
import logging
import warnings

warnings.filterwarnings("ignore")

logging.getLogger("torch").setLevel(logging.ERROR)

sys.stderr = open(os.devnull, 'w')
os.environ['PYTHONWARNINGS'] = 'ignore'

import os

def is_in_pwd(path):
    path = os.path.abspath(path)
    cwd = os.path.abspath(os.getcwd())

    try:
        return os.path.commonpath([path, cwd]) == cwd
    except ValueError:
        return False
```

Phase 1: Data Merging & Path Verification

```
import json
import os
import random
from collections import Counter
from tqdm.notebook import tqdm

# =====
# CONFIGURATION
# =====
TRAIN_JSON =
"/kaggle/input/datasets/aadarshprabhakumar/jsonfile/train.json"
VAL_JSON =
"/kaggle/input/datasets/aadarshprabhakumar/jsonfile/val.json"

TRAIN_IMG_DIR = "/kaggle/input/datasets/aadarshprabhakumar/dataset-
train/train"
```

```
VAL_IMG_DIR = "/kaggle/input/datasets/aadarshprabhakumar/dataset-validation"
```

```
DICT_PATH =  
"/kaggle/input/datasets/aadarshprabhakumar/dictionary/eng_hi.json"
```

```
OUTPUT_PATH = "main_dataset_en.json"
```

```
TEST_SPLIT_RATIO = 0.50
```

```
RANDOM_SEED = 42
```

```
# =====  
# STEP 0: Dictionary Verification  
# =====
```

```
def verify_dictionary(dict_path):  
    print("\n--- Dictionary Verification ---")  
  
    if not os.path.exists(dict_path):  
        print(f" Dictionary not found at {dict_path}.")  
        return None  
  
    with open(dict_path, 'r', encoding='utf-8') as f:  
        eng_hi_dict = json.load(f)  
  
    print(f" Dictionary loaded: {len(eng_hi_dict)} mappings")  
  
    items = list(eng_hi_dict.items())[:5]  
    for eng, hi in tqdm(items, desc="Inspecting dictionary",  
unit="pair"):  
        print(f"      '{eng}' -> '{hi}'")  
  
    return eng_hi_dict
```

```
# =====  
# STEP 1: Load JSON  
# =====
```

```
def load_vizwiz_json(filepath):  
    print(f"\nLoading: {filepath}")  
  
    with open(filepath, 'r') as f:  
        data = json.load(f)  
  
    print(f" -> Loaded {len(data)} samples")  
  
    # visual checkpoint (keeps tqdm UI alive in notebooks)  
    for _ in tqdm(range(1), desc="File loaded", leave=False):  
        pass  
  
    return data
```

```

# =====
# STEP 2: Path Injection
# =====
def inject_paths_and_tag(data, source_tag, img_dir):

    processed = []
    missing_count = 0
    dir_exists = os.path.exists(img_dir)

    for item in tqdm(data, desc=f"Injecting [{source_tag}]",
unit="img"):
        image_filename = item.get('image', '')
        full_path = os.path.join(img_dir, image_filename)

        path_exists = os.path.isfile(full_path) if dir_exists else
None

        if path_exists is False:
            missing_count += 1

        processed.append({
            'id': item.get('id'),
            'image': image_filename,
            'image_path': full_path,
            'source': source_tag,
            'question': item.get('question', ''),
            'answers': item.get('answers', []),
            'answer_type': item.get('answer_type', ''),
            'question_type': item.get('question_type', ''),
        })

    print(f"    -> Processed {len(processed)} | Missing images:
{missing_count}")
    return processed

# =====
# STEP 3: Stratified Split
# =====
def stratified_split(data, test_ratio=0.05, seed=42):

    random.seed(seed)

    type_groups = {}
    for item in data:
        type_groups.setdefault(item['answer_type'], []).append(item)

    train_list, test_list = [], []

```

```

    for atype, items in tqdm(type_groups.items(), desc="Stratifying",
unit="type"):
        random.shuffle(items)
        n_test = max(1, int(len(items) * test_ratio))
        test_list.extend(items[:n_test])
        train_list.extend(items[n_test:])

    random.shuffle(train_list)
    random.shuffle(test_list)

    return train_list, test_list

# =====
# STEP 4: Validation
# =====
def validate_and_report(data, split_name):

    print(f"\n{'='*50}")
    print(f" VALIDATION: {split_name} ({len(data)})")
    print(f"{'='*50}")

    type_counts = Counter()
    empty_q = 0
    malformed = 0

    for item in tqdm(data, desc=f"Validating [{split_name}]",
unit="img"):
        type_counts[item['answer_type']] += 1

        if not item['question'].strip():
            empty_q += 1

        for ans in item['answers']:
            if 'answer' not in ans or 'answer_confidence' not in ans:
                malformed += 1
                break

    for atype, count in type_counts.items():
        print(f"    {atype:20s}: {count}")

    print(f"    Empty questions: {empty_q}")
    print(f"    Malformed answers: {malformed}")
    print(f"    Sample path: {data[0]['image_path']}")

# =====
# STEP 5: Save JSON (with tqdm)
# =====
def save_json_with_progress(data, output_path):

```

```

print("\n--- Saving Dataset ---")

with open(output_path, 'w', encoding='utf-8') as f:
    f.write("[\n")

    for i, item in enumerate(
        tqdm(data, desc="Writing JSON", unit="obj")
    ):
        json.dump(item, f, ensure_ascii=False)

        if i != len(data) - 1:
            f.write(",\n")

    f.write("\n]")

# =====
# MAIN PIPELINE
# =====
def main():

    print("="*60)
    print(" VIZWIZ PHASE 1 PIPELINE")
    print("="*60)

    # -----
    # EXISTING DATASET CHECK
    # -----
    existing_paths = [
        "/kaggle/working/main_dataset_en.json",
        "/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-hi/main_dataset_en.json",
    ]

    existing_dataset = None

    for path in existing_paths:
        if os.path.exists(path):
            print(f"\nFound existing dataset:")
            print(f" -> {path}")

            with open(path, 'r', encoding='utf-8') as f:
                existing_dataset = json.load(f)

            print(f" -> Loaded {len(existing_dataset)} samples")

            if (not is_in_pwd(path)):
                save_json_with_progress(existing_dataset, OUTPUT_PATH)

```

```

        return existing_dataset
    else:
        print(f"{path} does not exist")

# -----
# BUILD DATASET
# -----
eng_hi_dict = verify_dictionary(DICT_PATH)

train_raw = load_vizwiz_json(TRAIN_JSON)
val_raw   = load_vizwiz_json(VAL_JSON)

train_processed = inject_paths_and_tag(train_raw, 'train',
TRAIN_IMG_DIR)
val_processed   = inject_paths_and_tag(val_raw, 'val',
VAL_IMG_DIR)

val_split, test_split = stratified_split(
    val_processed,
    TEST_SPLIT_RATIO,
    RANDOM_SEED
)

train_split = train_processed

for x in train_split:
    x['split'] = 'train'

for x in test_split:
    x['split'] = 'test'

for x in val_split:
    x['split'] = 'val'

final_data = train_split + val_split + test_split

validate_and_report(train_split, "TRAIN")
validate_and_report(val_split, "VAL")
validate_and_report(test_split, "TEST")

save_json_with_progress(final_data, OUTPUT_PATH)

print("\nDONE")
print(f"Saved -> {OUTPUT_PATH}")

return final_data

dataset = main()

```

```
=====
VIZWIZ PHASE 1 PIPELINE
=====
```

```
Found existing dataset:
-> /kaggle/working/main_dataset_en.json
-> Loaded 24842 samples
```

Phase 2: The Multilingual Translation Pipeline

```
import os
import json
import torch
import re
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from tqdm.notebook import tqdm

# =====
# CONFIGURATION
# =====
INPUT_PATH = "main_dataset_en.json"
DICT_PATH =
"/kaggle/input/datasets/aadarshprabhakumar/dictionary/eng_hi.json"

OUTPUT_PATH = "/kaggle/working/main_dataset_hi.json"
NOTEBOOK_TRANS_PATH = "/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-
hi/main_dataset_hi.json"

BATCH_SIZE = 64

# =====
# STEP 0: LOAD EXISTING TRANSLATED FILE IF FOUND
# =====

dataset = None
trans_path_existing = None

for trans_path in [OUTPUT_PATH, NOTEBOOK_TRANS_PATH]:
    print(f"Checking: {trans_path}")
    if os.path.exists(trans_path):
        trans_path_existing = trans_path

        print(f"Found existing translated file: {trans_path}")
        print("Loading existing dataset...")

        with open(trans_path, 'r', encoding='utf-8') as f:
            dataset = json.load(f)
```

```

        print("Loaded successfully.")

        break    # IMPORTANT

# =====
# ONLY RUN TRANSLATION IF NO FILE FOUND
# =====

if dataset is None:

    print("No translated dataset found.")
    print("Starting translation pipeline...")

    # =====
    # STEP 1: Load Dictionary & Dataset
    # =====

    with open(DICT_PATH, 'r', encoding='utf-8') as f:
        eng_hi_dict = json.load(f)

    with open(INPUT_PATH, 'r', encoding='utf-8') as f:
        dataset = json.load(f)

    # =====
    # STEP 2: MODEL SETUP
    # =====

    MODEL_NAME = "facebook/nllb-200-distilled-600M"

    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Using device: {device}")

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

    model = AutoModelForSeq2SeqLM.from_pretrained(
        MODEL_NAME,
        torch_dtype=torch.float16 if device == "cuda" else
torch.float32,
    )

    model = model.to(device)
    model.eval()

    tokenizer.src_lang = "eng_Latn"
    HINDI_TOKEN_ID = tokenizer.convert_tokens_to_ids("hin_Deva")

    print("Model loaded successfully")

    # =====
    # HELPER FUNCTIONS

```



```

# =====

def clean_input(text):

    if not text or not text.strip():
        return "unanswerable"

    text = re.sub(r'\s+', ' ', text).strip()
    return text

def apply_dictionary(text):

    words = text.split()
    processed_words = []

    for word in words:

        if word in eng_hi_dict:
            processed_words.append(eng_hi_dict[word][0])

        elif word.lower() in eng_hi_dict:
            processed_words.append(eng_hi_dict[word.lower()][0])

        else:
            processed_words.append(word)

    return " ".join(processed_words)

# =====
# TRANSLATE PIPELINE
# =====

def translate_pipeline(texts):

    texts = [clean_input(t) for t in texts]

    inputs = tokenizer(
        texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=128
    ).to(device)

    with torch.no_grad():

        outputs = model.generate(
            *inputs,
            forced_bos_token_id=HINDI_TOKEN_ID,
            max_new_tokens=128,

```

```

        num_beams=1,
        early_stopping=True,
        do_sample=False
    )

    translated = tokenizer.batch_decode(
        outputs,
        skip_special_tokens=True
    )

    final_outputs = []

    for t in translated:
        t = apply_dictionary(t)
        final_outputs.append(t)

    return final_outputs

# =====
# PASS 1: QUESTIONS
# =====

print("\n--- PASS 1: Translating Questions ---")

questions_to_translate = [
    item['question']
    for item in dataset
]

translated_questions = []

for i in tqdm(
    range(0, len(questions_to_translate), BATCH_SIZE),
    desc="Questions"
):

    batch = questions_to_translate[i:i+BATCH_SIZE]

    translated_questions.extend(
        translate_pipeline(batch)
    )

for i in range(len(dataset)):
    dataset[i]['question_hi'] = translated_questions[i]

# =====
# PASS 2: ANSWERS
# =====

print("\n--- PASS 2: Translating Answers ---")

```

```

all_answers_flat = []
index_mapping = []

for i, item in enumerate(dataset):
    for j, ans_obj in enumerate(item['answers']):
        all_answers_flat.append(
            ans_obj['answer']
        )
        index_mapping.append((i, j))

translated_answers_flat = []

for i in tqdm(
    range(0, len(all_answers_flat), BATCH_SIZE),
    desc="Answers"
):
    batch = all_answers_flat[i:i+BATCH_SIZE]

    translated_answers_flat.extend(
        translate_pipeline(batch)
    )

# =====
# PASS 3: REASSEMBLY
# =====

print("\n--- PASS 3: Reassembling JSON ---")

for idx, (i, j) in enumerate(
    tqdm(index_mapping, desc="Dictionary Pass")
):
    ans_hindi_final = translated_answers_flat[idx]

    if 'answers_hi' not in dataset[i]:
        dataset[i]['answers_hi'] = []

    dataset[i]['answers_hi'].append({
        'answer': ans_hindi_final,
        'answer_confidence': dataset[i]['answers'][j]
        ['answer_confidence']
    })

if (not is_in_pwd(trans_path_existing)):
    # =====
    # SAVE FINAL DATASET

```

```
# =====

print("\nSaving Hindi Dataset...")

with open(OUTPUT_PATH, 'w', encoding='utf-8') as f:
    json.dump(dataset, f, ensure_ascii=False, indent=4)

print(f"SUCCESS: Saved to {OUTPUT_PATH}")
```

```
Checking: /kaggle/working/main_dataset_hi.json
Found existing translated file: /kaggle/working/main_dataset_hi.json
Loading existing dataset...
Loaded successfully.
```

Phase 3: Model-A Architecture & Dataset Class

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import (
    CLIPVisionModel,
    CLIPImageProcessor,
    XLMRobertaModel,
    XLMRobertaTokenizer
)
from PIL import Image
import json

# =====
# CONFIGURATION
# =====

MODEL_CLIP = "openai/clip-vit-large-patch14"
MODEL_XLMR = "xlm-roberta-base"

TRAIN_JSON_HI = "main_dataset_hi.json"
TRAIN_JSON_EN = "main_dataset_en.json"

BATCH_SIZE = 8          # Conservative for T4; we'll use gradient
                           accumulation later
MAX_TEXT_LEN = 32
IMAGE_SIZE = 224

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# Answer Type Mapping for Classifier Head
ANS_TYPE_MAP = {'yes/no': 0, 'number': 1, 'other': 2, 'unanswerable':
3}
ANS_TYPE_MAP_REV = {v: k for k, v in ANS_TYPE_MAP.items()}

# =====
```

```

# STEP 1: The Dataset Class
# =====
class VizWizVQADataset(Dataset):
    def __init__(self, json_path, feature_extractor, tokenizer,
lang='hi'):
        print(f"Loading Dataset from {json_path}...")
        with open(json_path, 'r', encoding='utf-8') as f:
            self.data = json.load(f)

        self.feature_extractor = feature_extractor
        self.tokenizer = tokenizer
        self.lang = lang

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]

        # --- Image Processing ---
        try:
            image = Image.open(item['image_path']).convert("RGB")
            pixel_values = self.feature_extractor(images=image,
return_tensors="pt").pixel_values.squeeze(0)
        except Exception as e:
            # Fallback for missing/corrupt images: black image
            pixel_values = torch.zeros((3, IMAGE_SIZE, IMAGE_SIZE))

        # --- Text Processing ---
        if self.lang == 'hi':
            text = item.get('question_hi', item['question'])
        else:
            text = item['question']

        text_tokens = self.tokenizer(
            text,
            padding='max_length',
            truncation=True,
            max_length=MAX_TEXT_LEN,
            return_tensors="pt"
        )

        input_ids = text_tokens.input_ids.squeeze(0)
        attention_mask = text_tokens.attention_mask.squeeze(0)

        # --- Labels ---
        answer_type_str = item['answer_type']
        answer_type_label = ANS_TYPE_MAP.get(answer_type_str, 3) #
Default to unanswerable

```

```

        return {
            'pixel_values': pixel_values,
            'input_ids': input_ids,
            'attention_mask': attention_mask,
            'answer_type_label': torch.tensor(answer_type_label,
dtype=torch.long),
        }

# =====
# STEP 2: The Cross-Attention Fusion Module
# =====
class CrossAttentionFusion(nn.Module):
    def __init__(self, text_dim=768, image_dim=1024, num_heads=8):
        super().__init__()
        # Project image dimension (1024) to match text dimension (768)
        self.img_proj = nn.Linear(image_dim, text_dim)

        # Multi-Head Attention: Text (Query) attends to Image (Key,
Value)
        self.cross_attn = nn.MultiheadAttention(embed_dim=text_dim,
num_heads=num_heads, batch_first=True)

        # Layer Norm for stability
        self.layer_norm = nn.LayerNorm(text_dim)

    def forward(self, text_features, image_features):
        """
        text_features: [Batch, Seq_Len, 768]
        image_features: [Batch, Num_Patches, 1024]
        """
        # Project image to text dimension
        img_projected = self.img_proj(image_features) # [Batch,
Num_Patches, 768]

        # Cross-Attention: Text asks, Image answers
        attn_output, _ = self.cross_attn(
            query=text_features,
            key=img_projected,
            value=img_projected
        )

        # Residual connection & LayerNorm
        fused_features = self.layer_norm(text_features + attn_output)

        return fused_features

# =====
# STEP 3: The Complete Model A
# =====
class ModelA_TypeExpert(nn.Module):

```

```

def __init__(self):
    super().__init__()

    print("Initializing Model A: ViT-L/14 + XLM-RoBERTa +
CrossAttention")

    # 1. Vision Backbone (Loaded in FP16 to save VRAM)
    self.vision_model =
CLIPVisionModel.from_pretrained(MODEL_CLIP, torch_dtype=torch.float16)

    # 2. Text Backbone (Loaded in FP16 to save VRAM)
    self.text_model = XLMRobertaModel.from_pretrained(MODEL_XLMR,
torch_dtype=torch.float16)

    # 3. Fusion Layer (Will be trained in FP32 for stability)
    self.fusion = CrossAttentionFusion(text_dim=768,
image_dim=1024, num_heads=8)

    # 4. Classifier Head (Answer Type: 4 classes)
    self.classifier = nn.Sequential(
        nn.Linear(768, 256),
        nn.ReLU(),
        nn.Dropout(0.2),
        nn.Linear(256, 4)
    )

    # 5. Generator Head placeholder (Will be added in Phase 4.2)
    self.generator = None

def forward(self, pixel_values, input_ids, attention_mask):
    # Extract Vision Features
    vision_outputs = self.vision_model(pixel_values=pixel_values)
    # Get patch embeddings (excluding CLS token at index 0 for
pure patch representation)
    image_features = vision_outputs.last_hidden_state[:, 1:, :]

    # Extract Text Features
    text_outputs = self.text_model(input_ids=input_ids,
attention_mask=attention_mask)
    text_features = text_outputs.last_hidden_state

    # Fuse via Cross-Attention
    fused_features = self.fusion(text_features, image_features)

    # We take the <s> (CLS equivalent) token representation for
classification
    cls_representation = fused_features[:, 0, :]

    # Predict Answer Type

```

```

        type_logits = self.classifier(cls_representation)

        return type_logits

# =====
# STEP 4: Initialization & VRAM Check
# =====
print("Loading Tokenizers and Feature Extractors...")
feature_extractor = CLIPImageProcessor.from_pretrained(MODEL_CLIP)
tokenizer = XLMRobertaTokenizer.from_pretrained(MODEL_XLMR)

print("\nInstantiating Model A...")
model_a = ModelA_TypeExpert().to(DEVICE)

# Quick VRAM check
if torch.cuda.is_available():
    alloc = torch.cuda.memory_allocated() / (1024**3)
    reserved = torch.cuda.memory_reserved() / (1024**3)
    print(f"\nGPU VRAM Status after loading Model A:")
    print(f"    Allocated: {alloc:.2f} GB")
    print(f"    Reserved: {reserved:.2f} GB")
    print(f"    T4 Total: 16.00 GB")

# Quick DataLoaders test
print("\nTesting Dataset Loading...")
test_dataset = VizWizVQADataset(TRAIN_JSON_HI, feature_extractor,
                                tokenizer, lang='hi')
test_sample = test_dataset[0]
print(f"Sample pixel_values shape: {test_sample['pixel_values'].shape}")
print(f"Sample input_ids shape: {test_sample['input_ids'].shape}")
print(f"Sample answer_type_label: {test_sample['answer_type_label']}")

print("\nPhase 3 Architecture Built Successfully!")

Loading Tokenizers and Feature Extractors...

Instantiating Model A...
Initializing Model A: ViT-L/14 + XLM-RoBERTa + CrossAttention

{"model_id": "c218c1361a544e53bb3af6cb4931b354", "version_major": 2, "version_minor": 0}

{"model_id": "4e8cd6d7accf47cd924e561aa6997cd0", "version_major": 2, "version_minor": 0}

GPU VRAM Status after loading Model A:
    Allocated: 1.10 GB
    Reserved: 1.12 GB
    T4 Total: 16.00 GB

```



```
Testing Dataset Loading...
Loading Dataset from main_dataset_hi.json...
Sample pixel_values shape: torch.Size([3, 224, 224])
Sample input_ids shape: torch.Size([32])
Sample answer_type_label: 2
```

Phase 3 Architecture Built Successfully!

Phase 4.1: Training the Fusion and Classifier Heads

```
import shutil
import os

def copy_checkpoint_to_working(source_path, destination_path):

    if os.path.exists(source_path):

        shutil.copy(source_path, destination_path)

        print("Checkpoint copied successfully!")
        print(f"Destination: {destination_path}")

    else:

        print("Source checkpoint not found.")

import os
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Subset
from transformers import get_cosine_schedule_with_warmup
from tqdm.notebook import tqdm
import numpy as np
import math

# =====
# CONFIGURATION
# =====
EPOCHS = 20
PATIENCE = 5
LEARNING_RATE = 1e-4
BATCH_SIZE = 16
GRAD_ACCUM_STEPS = 2
WEIGHT_DECAY = 0.01

MODEL_A_STORE_PATH = "/kaggle/working/model_a_phase1_best.pt"
MODEL_A_NB_PATH =
"/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-hi/model_a_phase1_best.
pt"
```

```

# =====
# CHECKPOINT FUNCTION
# =====
def load_checkpoint_if_exists(model, paths, device):
    for path in paths:
        if os.path.exists(path):
            model.load_state_dict(torch.load(path,
map_location=device), strict=False)
            print(f"Loaded checkpoint from:\n{path}")

            if not is_in_pwd(path):
                copy_checkpoint_to_working(path, MODEL_A_STORE_PATH)

    return path
    print("No checkpoint found. Training from scratch.")
    return None

# =====
# VALIDATION FUNCTION
# =====
def validate_model(model, val_loader, val_dataset, device):

    model.eval()

    val_type_correct = 0
    val_total = 0
    val_ans_score_total = 0.0

    with torch.no_grad():

        for batch in tqdm(val_loader, desc="Validation"):

            pixel_values = batch['pixel_values'].to(device)
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['answer_type_label'].to(device)

            with torch.cuda.amp.autocast():

                logits = model(
                    pixel_values,
                    input_ids,
                    attention_mask
                )

            preds = torch.argmax(logits, dim=1)

            val_type_correct += (preds == labels).sum().item()
            val_total += labels.size(0)

```

```

        for p, l in zip(preds, labels):
            if p == l:
                val_ans_score_total += min(1, 1 / 3)

    val_type_acc = val_type_correct / val_total
    avg_val_ans_score = val_ans_score_total / len(val_dataset)

    return val_type_acc, avg_val_ans_score

# =====
# STEP 1: DataLoaders Preparation
# =====
print("Preparing Datasets & DataLoaders...")

full_dataset_hi = VizWizVQADataset(TRAIN_JSON_HI, feature_extractor,
tokenizer, lang='hi')

train_indices = [i for i, x in enumerate(full_dataset_hi.data) if
x['split'] == 'train']
val_indices = [i for i, x in enumerate(full_dataset_hi.data) if
x['split'] == 'val']
test_indices = [i for i, x in enumerate(full_dataset_hi.data) if
x['split'] == 'test']

train_dataset = Subset(full_dataset_hi, train_indices)
val_dataset = Subset(full_dataset_hi, val_indices)
test_dataset = Subset(full_dataset_hi, test_indices)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True)

print(f"Train size: {len(train_dataset)} | Val size:
{len(val_dataset)} | Test size: {len(test_dataset)}")

# =====
# STEP 2: Fusion Module
# =====
class CrossAttentionFusionV2(nn.Module):
    def __init__(self, text_dim=768, image_dim=1024, num_heads=8):
        super().__init__()
        self.img_proj = nn.Linear(image_dim, text_dim)
        self.scale = math.sqrt(text_dim)
        self.cross_attn = nn.MultiheadAttention(embed_dim=text_dim,
num_heads=num_heads, batch_first=True, dropout=0.3)

```

```

        self.layer_norm = nn.LayerNorm(text_dim)
        self.fusion_dropout = nn.Dropout(0.3)

    def forward(self, text_features, image_features):
        img_projected = self.img_proj(image_features) / self.scale
        attn_output, _ = self.cross_attn(query=text_features,
key=img_projected, value=img_projected)
        fused_features = self.layer_norm(text_features + attn_output)
        fused_features = self.fusion_dropout(fused_features)
        return fused_features

# =====
# STEP 3: Model Setup
# =====
print("\nRe-initializing Model A...")
model_a.fusion = CrossAttentionFusionV2(text_dim=768, image_dim=1024,
num_heads=8).to(DEVICE)
model_a.classifier = nn.Sequential(
    nn.Linear(768, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 4)
).to(DEVICE)

# Load Checkpoint
checkpoint_loaded = load_checkpoint_if_exists(model_a,
[MODEL_A_STORE_PATH, MODEL_A_NB_PATH], DEVICE)

# =====
# SKIP TRAINING IF CHECKPOINT EXISTS
# =====
if checkpoint_loaded is not None:

    model_a.eval()

    print("\nCheckpoint already exists.\nSkipping Phase 4.1
Training.")
    print("Running validation...")

    val_type_acc, avg_val_ans_score = validate_model(
        model_a,
        val_loader,
        val_dataset,
        DEVICE
    )

    print(f"\nValidation Results")
    print(f"Val Type Acc: {val_type_acc:.4f}")
    print(f"Proxy Val ANS Score: {avg_val_ans_score:.4f}")

```

```

else:
    # Freeze Vision Model
    for param in model_a.vision_model.parameters():
        param.requires_grad = False
    # Freeze Text Model
    for param in model_a.text_model.parameters():
        param.requires_grad = False

    trainable_params = sum(p.numel() for p in model_a.parameters() if
p.requires_grad)
    total_params = sum(p.numel() for p in model_a.parameters())
    print(f"Trainable Parameters: {trainable_params:,} /
{total_params:,}")

    # Optimizer & Scheduler
    decay_params = [p for n, p in model_a.named_parameters() if
p.requires_grad and 'weight' in n and 'layer_norm' not in n]
    no_decay_params = [p for n, p in model_a.named_parameters() if
p.requires_grad and ('bias' in n or 'layer_norm' in n)]

    optimizer = torch.optim.AdamW([
        {'params': decay_params, 'weight_decay': WEIGHT_DECAY},
        {'params': no_decay_params, 'weight_decay': 0.0}
    ], lr=LEARNING_RATE)

    scheduler = get_cosine_schedule_with_warmup(optimizer,
num_warmup_steps=int(0.1 * len(train_loader) * EPOCHS),
num_training_steps=len(train_loader) * EPOCHS)
    criterion = nn.CrossEntropyLoss()

    # TRAINING LOOP
    best_val_acc = 0
    epochs_no_improve = 0
    print(f"\nStarting Phase 4.1 Training for {EPOCHS} epochs...")

    for epoch in range(EPOCHS):
        model_a.train()
        model_a.vision_model.eval()
        model_a.text_model.eval()

        running_loss = 0.0
        train_type_correct = 0
        train_total = 0
        optimizer.zero_grad()

        pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}
[Train]")
        for batch_idx, batch in enumerate(pbar):
            pixel_values = batch['pixel_values'].to(DEVICE)

```

```

        input_ids = batch['input_ids'].to(DEVICE)
        attention_mask = batch['attention_mask'].to(DEVICE)
        labels = batch['answer_type_label'].to(DEVICE)

        with torch.cuda.amp.autocast():
            logits = model_a(pixel_values, input_ids,
attention_mask)
            loss = criterion(logits, labels)

        loss = loss / GRAD_ACCUM_STEPS
        loss.backward()

        if (batch_idx + 1) % GRAD_ACCUM_STEPS == 0:
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

        running_loss += loss.item() * GRAD_ACCUM_STEPS
        preds = torch.argmax(logits, dim=1)
        train_type_correct += (preds == labels).sum().item()
        train_total += labels.size(0)
        pbar.set_postfix({'loss':
f'{running_loss/(batch_idx+1):.4f}', 'type_acc':
f'{train_type_correct/train_total:.4f}'})

    # VALIDATION
    model_a.eval()
    val_type_acc, avg_val_ans_score = validate_model(
        model_a,
        val_loader,
        val_dataset,
        DEVICE
    )

    print(f"--- Epoch {epoch+1} Summary ---")
    print(f"Val Type Acc: {val_type_acc:.4f} | Proxy Val ANS
Score: {avg_val_ans_score:.4f}")

    if val_type_acc > best_val_acc:
        best_val_acc = val_type_acc
        epochs_no_improve = 0
        torch.save(model_a.state_dict(), MODEL_A_STORE_PATH)
        print("Model improved and saved!")
    else:
        epochs_no_improve += 1
        print(f"No improvement for {epochs_no_improve} epoch(s).")

    if epochs_no_improve >= PATIENCE:
        print(f"Early stopping triggered at epoch {epoch+1}!")
        break

```

```

    print(f"\nPhase 4.1 Training Complete. Best Val Type Acc:
{best_val_acc}")

Preparing Datasets & DataLoaders...
Loading Dataset from main_dataset_hi.json...
Train size: 20523 | Val size: 2160 | Test size: 2159

Re-initializing Model A...
Loaded checkpoint from:
/kaggle/working/model_a_phase1_best.pt

Checkpoint already exists.
Skipping Phase 4.1 Training.
Running validation...

{"model_id": "7ba7c946b2214a59841403f94ce72b10", "version_major": 2, "version_minor": 0}

Validation Results
Val Type Acc: 0.6319
Proxy Val ANS Score: 0.2106

```

Evaluation 1

```

%%capture
!pip install ipywidgets

import os
import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    classification_report, ConfusionMatrixDisplay,
    precision_score, recall_score, f1_score
)
from tqdm.auto import tqdm
from collections import defaultdict

# Ensure model_a and DEVICE are defined in your environment before
running
model = model_a
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# -----
# CORE EVALUATION FUNCTION
# -----
def run_full_evaluation(model, loader, dataset_name, base_save_dir):

```

```

"""
    Runs evaluation using the exact architectural logic of
    run_validation_1.
"""
save_path = os.path.join(base_save_dir, dataset_name)
os.makedirs(save_path, exist_ok=True)

print(f"\n Starting Evaluation: {dataset_name.upper()}")
model.eval()

all_preds, all_labels = [], []
target_names = ['yes/no', 'number', 'other', 'unanswerable']

# VQA-style scoring accumulators (Matching run_validation_1 logic)
vqa_ans_score_total = 0.0
vqa_total = 0
per_class_vqa_score = defaultdict(float)
per_class_count = defaultdict(int)

with torch.no_grad():
    # tqdm wrapping the loader
    for batch in tqdm(loader, desc=f"Evaluating {dataset_name}",
leave=True):
        pixel_values = batch['pixel_values'].to(DEVICE)
        input_ids = batch['input_ids'].to(DEVICE)
        attention_mask = batch['attention_mask'].to(DEVICE)
        # Support both common key names
        type_labels = batch.get('type_labels',
batch.get('answer_type_label')).to(DEVICE)

        with torch.cuda.amp.autocast():
            # --- MANUAL FEATURE EXTRACTION (Same as
run_validation_1) ---
            vision_outputs =
model.vision_model(pixel_values=pixel_values)
            image_features = vision_outputs.last_hidden_state[:,
1:, :]

            text_outputs = model.text_model(input_ids=input_ids,
attention_mask=attention_mask)
            text_features = text_outputs.last_hidden_state

            # --- FUSION & CLASSIFICATION ---
            fused_features = model.fusion(text_features,
image_features)
            cls_representation = fused_features[:, 0, :]
            type_logits = model.classifier(cls_representation)

            preds = torch.argmax(type_logits, dim=1)
            preds_np = preds.cpu().numpy()

```



```

        labels_np = type_labels.cpu().numpy()

        all_preds.extend(preds_np)
        all_labels.extend(labels_np)

        # VQA-style score: Correct Type = 1 match. Score = min(1,
matches/3)
        # In type-only mode, this effectively becomes accuracy *
(1/3)
        for p, l in zip(preds_np, labels_np):
            vqa_total += 1
            per_class_count[int(l)] += 1
            if p == l:
                score = 1/3
                vqa_ans_score_total += score
                per_class_vqa_score[int(l)] += score

        # -----
        # METRICS CALCULATION
        # -----
        avg_vqa_ans_score = vqa_ans_score_total / vqa_total if vqa_total >
0 else 0.0

        # 1. Get dictionary for saving to CSV
        report_dict = classification_report(
            all_labels, all_preds, target_names=target_names,
output_dict=True, zero_division=0
        )

        # 2. PRINT THE REPORT TO CONSOLE (This is what was missing!)
        print("\n" + "="*60)
        print(f" CLASSIFICATION REPORT: {dataset_name.upper()}")
        print("="*60)
        print(classification_report(all_labels, all_preds,
target_names=target_names, zero_division=0))
        print(f" VQA Score (Mean): {avg_vqa_ans_score:.4f}")
        print("="*60 + "\n")

        # Per-Class VQA Score Calculation
        per_class_vqa = {name: (per_class_vqa_score[i] /
per_class_count[i] if per_class_count[i] > 0 else 0.0)
                        for i, name in enumerate(target_names)}

        # -----
        # SAVE & EXPORT
        # -----
        # 1. Classification Report CSV
        df_report = pd.DataFrame(report_dict).transpose()
        df_report.to_csv(f"{save_path}/classification_report.csv")

```

```

# 2. Summary Metrics CSV
summary = pd.DataFrame({
    "Metric": ["Accuracy", "VQA Score (Mean)", "Macro F1",
"Weighted F1"],
    "Value": [report_dict["accuracy"], avg_vqa_ans_score,
report_dict["macro avg"]["f1-score"], report_dict["weighted avg"]["f1-
score"]]
})
summary.to_csv(f"{save_path}/summary_metrics.csv", index=False)

# 3. Visual Dashboard
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# F1 Bar Chart
axes[0, 0].bar(target_names, [report_dict[n]['f1-score'] for n in
target_names], color='#C44E52')
axes[0, 0].set_title(f"F1-score per Class ({dataset_name})")
axes[0, 0].set_ylim(0, 1)

# VQA Score Bar Chart
axes[0, 1].bar(target_names, [per_class_vqa[n] for n in
target_names], color='#DD8452')
axes[0, 1].set_title("VQA Score per Class")
axes[0, 1].set_ylim(0, 0.4) # Max possible is 0.33

# Support (Sample Distribution)
axes[1, 0].bar(target_names, [report_dict[n]['support'] for n in
target_names], color='#8172B2')
axes[1, 0].set_title("Support per Class")

# Confusion Matrix
ConfusionMatrixDisplay.from_predictions(all_labels, all_preds,
display_labels=target_names, cmap="Blues", ax=axes[1, 1])
axes[1, 1].set_title("Confusion Matrix")

plt.tight_layout()
plt.savefig(f"{save_path}/evaluation_dashboard.png")
plt.close()

print(f"--- {dataset_name.capitalize()} Evaluation Complete ---")
return report_dict["accuracy"]

# -----
# MAIN EXECUTION BLOCK
# -----
BASE_SAVE_DIR = "evaluations/model_a_base"

# 1. Run on Val
val_accuracy = run_full_evaluation(model, val_loader, "validation",
BASE_SAVE_DIR)

```

2. Run on Test

```
test_accuracy = run_full_evaluation(model, test_loader, "test",
BASE_SAVE_DIR)
```

```
print(f"\nFinal Comparison:")
print(f"Validation Accuracy: {val_accuracy:.4f}")
print(f"Test Accuracy:      {test_accuracy:.4f}")
```

Starting Evaluation: VALIDATION

```
{"model_id": "ac78c4505659442da175f4a95af86027", "version_major": 2, "version_minor": 0}
```

CLASSIFICATION REPORT: VALIDATION

	precision	recall	f1-score	support
yes/no	0.00	0.00	0.00	96
number	0.00	0.00	0.00	19
other	0.69	0.82	0.75	1329
unanswerable	0.47	0.38	0.42	716
accuracy			0.63	2160
macro avg	0.29	0.30	0.29	2160
weighted avg	0.58	0.63	0.60	2160

VQA Score (Mean): 0.2106

--- Validation Evaluation Complete ---

Starting Evaluation: TEST

```
{"model_id": "82f624b7c72146f7a357328b4efd5be1", "version_major": 2, "version_minor": 0}
```

CLASSIFICATION REPORT: TEST

	precision	recall	f1-score	support
yes/no	0.00	0.00	0.00	99
number	0.00	0.00	0.00	29
other	0.70	0.84	0.76	1362
unanswerable	0.45	0.36	0.40	669

accuracy			0.64	2159
macro avg	0.29	0.30	0.29	2159
weighted avg	0.58	0.64	0.61	2159

VQA Score (Mean): 0.2131

=====

--- Test Evaluation Complete ---

Final Comparison:

Validation Accuracy: 0.6319

Test Accuracy: 0.6392

Phase 4.2: Full Fine-Tuning & The Generative Head

```
import os
import torch
import torch.nn as nn
from transformers import GPT2LMHeadModel, GPT2Tokenizer
from tqdm.notebook import tqdm

# =====
# CONFIGURATION
# =====
STAB_EPOCHS = 2
FULL_EPOCHS = 4
LEARNING_RATE = 2e-5
BATCH_SIZE = 4
GRAD_ACCUM = 8
MAX_ANS_LEN = 10
MAX_GRAD_NORM = 1.0    # Strict gradient clipping

MODEL_A_FULL_STORE_PATH = "/kaggle/working/model_a_full_best.pt"
MODEL_A_FULL_NB_PATH = "/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-
hi/model_a_full_best.pt"

MODEL_A_STORE_PATH = "/kaggle/working/model_a_phase1_best.pt"
MODEL_A_NB_PATH =
"/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-hi/model_a_phase1_best.
pt"

# =====
# STEP 1: Generative Head Definition
# =====
print("Adding Generative GPT-2 Head to Model A...")

gen_tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
gen_tokenizer.pad_token = gen_tokenizer.eos_token

class GeneratorHead(nn.Module):
```

```

def __init__(self, text_dim=768):
    super().__init__()
    self.gpt2 = GPT2LMHeadModel.from_pretrained("gpt2")
    self.proj = nn.Linear(text_dim, self.gpt2.config.n_embd)

    def forward(self, fused_cls_representation, answer_input_ids=None,
labels=None):
        gpt_input_proj =
self.proj(fused_cls_representation).unsqueeze(1)

        if answer_input_ids is not None:
            answer_embeddings =
self.gpt2.transformer.wte(answer_input_ids)
            inputs_embeddings = torch.cat([gpt_input_proj,
answer_embeddings[:, :-1, :]], dim=1)
        else:
            inputs_embeddings = gpt_input_proj

        outputs = self.gpt2(inputs_embeddings=inputs_embeddings,
labels=labels)
        return outputs.loss if labels is not None else outputs.logits

model_a.generator = GeneratorHead().to(DEVICE)

# =====
# STEP 2: Checkpoint Loading Logic
# =====
def load_full_checkpoint_if_exists(model, paths, device):
    for path in paths:
        if os.path.exists(path):
            model.load_state_dict(torch.load(path,
map_location=device))
            print(f"Loaded Full Model checkpoint from:\n{path}")

            if not is_in_pwd(path):
                torch.save(model_a.state_dict(),
MODEL_A_FULL_STORE_PATH)
                print("Full Model checkpoint saved to",
MODEL_A_STORE_PATH)

            return path
    print("No Full Model checkpoint found. Will train from Phase 4.1
weights.")
    return None

checkpoint_loaded = load_full_checkpoint_if_exists(
    model_a,
    [MODEL_A_FULL_STORE_PATH, MODEL_A_FULL_NB_PATH],
    DEVICE
)

```

```

# =====
# STEP 3: Dataset & Collate
# =====
def collate_fn_phase2(batch):
    pixel_values = torch.stack([b['pixel_values'] for b in batch])
    input_ids = torch.stack([b['input_ids'] for b in batch])
    attention_mask = torch.stack([b['attention_mask'] for b in batch])
    type_labels = torch.stack([b['answer_type_label'] for b in batch])

    ans_texts = []
    orig_idx = []
    for b in batch:
        idx = b.get('orig_idx', 0)
        orig_idx.append(idx)
        ans_texts.append(full_dataset_hi.data[idx]['answers_hi'][0]
['answer'])

    gen_tokens = gen_tokenizer(ans_texts, return_tensors="pt",
padding=True, truncation=True, max_length=MAX_ANS_LEN)
    gen_labels = gen_tokens.input_ids.clone()
    gen_labels[gen_labels == gen_tokenizer.pad_token_id] = -100

    return {
        'pixel_values': pixel_values, 'input_ids': input_ids,
        'attention_mask': attention_mask,
        'type_labels': type_labels, 'answer_input_ids':
gen_tokens.input_ids,
        'gen_labels': gen_labels, 'orig_idx': orig_idx
    }

for i in range(len(full_dataset_hi)):
    full_dataset_hi.data[i]['orig_idx'] = i

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2, pin_memory=True,
collate_fn=collate_fn_phase2)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True,
collate_fn=collate_fn_phase2)

criterion_type = nn.CrossEntropyLoss()

# =====
# Validation Helper
# =====
def run_validation(model, loader):
    model.eval()
    val_type_correct = 0
    val_total = 0

```

```

val_ans_score_total = 0.0
with torch.no_grad():
    for batch in tqdm(loader, desc="[Val]", leave=False):
        pixel_values = batch['pixel_values'].to(DEVICE)
        input_ids = batch['input_ids'].to(DEVICE)
        attention_mask = batch['attention_mask'].to(DEVICE)
        type_labels = batch['type_labels'].to(DEVICE)
        orig_idxxs = batch['orig_idxxs']

        with torch.cuda.amp.autocast():
            vision_outputs =
model.vision_model(pixel_values=pixel_values)
            image_features = vision_outputs.last_hidden_state[:,
1:, :]

            text_outputs = model.text_model(input_ids=input_ids,
attention_mask=attention_mask)
            text_features = text_outputs.last_hidden_state

            fused_features = model.fusion(text_features,
image_features)
            cls_representation = fused_features[:, 0, :]

            type_logits = model.classifier(cls_representation)
            gen_logits = model.generator(cls_representation)

            preds_type = torch.argmax(type_logits, dim=1)
            val_type_correct += (preds_type ==
type_labels).sum().item()
            val_total += type_labels.size(0)

            pred_ans_ids = torch.argmax(gen_logits, dim=-1)
            pred_ans_texts = gen_tokenizer.batch_decode(pred_ans_ids,
skip_special_tokens=True)

            for i, (pred_text, idx) in enumerate(zip(pred_ans_texts,
orig_idxxs)):
                gt_answers = [full_dataset_hi.data[idx]['answers_hi']
[j]['answer'] for j in range(min(3, len(full_dataset_hi.data[idx]
['answers_hi'])))]
                matches = sum(1 for gt in gt_answers if
pred_text.strip().lower() == gt.strip().lower())
                val_ans_score_total += min(1, matches / 3)

            val_type_acc = val_type_correct / val_total
            avg_val_ans_score = val_ans_score_total / val_total
            return val_type_acc, avg_val_ans_score

# =====
# SKIP TRAINING IF CHECKPOINT EXISTS

```

```

# =====
if checkpoint_loaded is not None:
    print("\nFull Model checkpoint already exists. Skipping Phase
4.2aTraining.")
    val_type_acc, avg_val_ans_score = run_validation(model_a,
val_loader)
    print(f"Val Type Acc: {val_type_acc:.4f} | REAL Val ANS Score:
{avg_val_ans_score:.4f}")
else:
    # Ensure starting from pristine Phase 4.1 weights
    print("Loading pristine Phase 4.1 weights to start Phase 4.2...")
    phasel_loaded = False
    for path in [MODEL_A_STORE_PATH, MODEL_A_NB_PATH]:
        if os.path.exists(path):
            model_a.load_state_dict(torch.load(path,
map_location=DEVICE), strict=False)
            print(f"Loaded Phase 4.1 checkpoint from: {path}")

            if not is_in_pwd(path):
                best_val_type_acc = val_type_acc
                torch.save(model_a.state_dict(),
MODEL_A_FULL_STORE_PATH)
                print("Full Model checkpoint saved to",
MODEL_A_STORE_PATH)

                phasel_loaded = True
                break
    if not phasel_loaded:
        raise FileNotFoundError("Phase 4.1 checkpoint missing! Cannot
stabilize.")

    # =====
    # PHASE 4.2a: STABILIZATION
    # =====
    print("\n--- PHASE 4.2a: Stabilizing Classifier & Generator Heads
---")
    for param in model_a.vision_model.parameters():
        param.requires_grad = False
    for param in model_a.text_model.parameters(): param.requires_grad
= False
    for param in model_a.fusion.parameters(): param.requires_grad =
False

    optimizer_stab = torch.optim.AdamW([
        {'params': model_a.classifier.parameters(), 'lr': 1e-4},
        {'params': model_a.generator.parameters(), 'lr': 1e-4}
    ], weight_decay=0.01)

    for epoch in range(STAB_EPOCHS):

```



```

        model_a.train()
        model_a.vision_model.eval(); model_a.text_model.eval();
model_a.fusion.eval()

        running_loss = 0; optimizer_stab.zero_grad()
        pbar = tqdm(train_loader, desc=f"Stab Epoch
{epoch+1}/{STAB_EPOCHS}")
        for batch_idx, batch in enumerate(pbar):
            pixel_values = batch['pixel_values'].to(DEVICE)
            input_ids = batch['input_ids'].to(DEVICE)
            attention_mask = batch['attention_mask'].to(DEVICE)
            type_labels = batch['type_labels'].to(DEVICE)
            answer_input_ids = batch['answer_input_ids'].to(DEVICE)
            gen_labels = batch['gen_labels'].to(DEVICE)

            with torch.cuda.amp.autocast():
                with torch.no_grad():
                    vision_outputs =
model_a.vision_model(pixel_values=pixel_values)
                    image_features =
vision_outputs.last_hidden_state[:, 1:, :]
                    text_outputs =
model_a.text_model(input_ids=input_ids, attention_mask=attention_mask)
                    text_features = text_outputs.last_hidden_state
                    fused_features = model_a.fusion(text_features,
image_features)
                    cls_detached = fused_features[:, 0, :].detach()

                    type_logits = model_a.classifier(cls_detached)
                    loss_type = criterion_type(type_logits, type_labels)

                    loss_gen = model_a.generator(cls_detached,
answer_input_ids=answer_input_ids, labels=gen_labels)
                    loss = loss_type + (0.5 * loss_gen)

                loss = loss / GRAD_ACCUM
                loss.backward()
                if (batch_idx + 1) % GRAD_ACCUM == 0:
optimizer_stab.step(); optimizer_stab.zero_grad()
                running_loss += loss.item() * GRAD_ACCUM
                pbar.set_postfix({'loss':
f'{running_loss/(batch_idx+1):.4f}'})

        val_type_acc, avg_val_ans_score = run_validation(model_a,
val_loader)
        print(f"Stabilization Val Type Acc: {val_type_acc:.4f} | Val ANS
Score: {avg_val_ans_score:.4f}")

```

Adding Generative GPT-2 Head to Model A...

```
{"model_id": "431949a07ce04e198147daaae0f40992", "version_major": 2, "version_minor": 0}
```

Loaded Full Model checkpoint from:
/kaggle/working/model_a_full_best.pt

Full Model checkpoint already exists. Skipping Phase 4.2a Training.

```
{"model_id": "", "version_major": 2, "version_minor": 0}
```

Val Type Acc: 0.6782 | REAL Val ANS Score: 0.0000

```
if checkpoint_loaded is not None:
    print("\nFull Model checkpoint already exists. Skipping Phase 4.2b Training.")
else:
```

```
    # =====
    # PHASE 4.2b: HIGHLY CONSTRAINED FULL FINE-TUNING
    # =====
```

```
    print("\n--- PHASE 4.2b: Constrained Full Fine-Tuning ---")

    # 1. Keep Vision Model completely FROZEN (visual features are
    # language agnostic)
    for param in model_a.vision_model.parameters():
        param.requires_grad = False
```

```
    # 2. Unfreeze Text and Fusion carefully
    for param in model_a.text_model.parameters(): param.requires_grad = True
    for param in model_a.fusion.parameters(): param.requires_grad = True
    for param in model_a.classifier.parameters(): param.requires_grad = True
    for param in model_a.generator.parameters(): param.requires_grad = True
```

```
    model_a.float()
```

```
    # 3. Conservative Differential Learning Rates
    optimizer_full = torch.optim.AdamW([
        {'params': model_a.text_model.parameters(), 'lr': 5e-6},
    # Very slow for RoBERTa
        {'params': model_a.fusion.parameters(), 'lr': 1e-5},
    # Slow for Fusion
        {'params': model_a.classifier.parameters(), 'lr': 2e-5},
    # Normal for Classifier
        {'params': model_a.generator.parameters(), 'lr': 2e-5},
    # Normal for Generator
    ], weight_decay=0.01)
```

```

# Initialize Scaler
scaler = torch.cuda.amp.GradScaler(enabled=True)

best_val_type_acc = 0

for epoch in range(FULL_EPOCHS):
    model_a.train()
    model_a.vision_model.eval() # Ensure ViT stays in eval mode

    running_loss = 0; running_type_loss = 0; running_gen_loss = 0
    optimizer_full.zero_grad()

    pbar = tqdm(train_loader, desc=f"Full Epoch
{epoch+1}/{FULL_EPOCHS}")
    for batch_idx, batch in enumerate(pbar):
        pixel_values = batch['pixel_values'].to(DEVICE)
        input_ids = batch['input_ids'].to(DEVICE)
        attention_mask = batch['attention_mask'].to(DEVICE)
        type_labels = batch['type_labels'].to(DEVICE)
        answer_input_ids = batch['answer_input_ids'].to(DEVICE)
        gen_labels = batch['gen_labels'].to(DEVICE)

        with torch.cuda.amp.autocast():
            vision_outputs =
model_a.vision_model(pixel_values=pixel_values)
            image_features = vision_outputs.last_hidden_state[:,
1:, :]

            text_outputs = model_a.text_model(input_ids=input_ids,
attention_mask=attention_mask)
            text_features = text_outputs.last_hidden_state

            fused_features = model_a.fusion(text_features,
image_features)
            cls_representation = fused_features[:, 0, :]

            type_logits = model_a.classifier(cls_representation)
            loss_type = criterion_type(type_logits.float(),
type_labels)

            loss_gen = model_a.generator(cls_representation,
answer_input_ids=answer_input_ids, labels=gen_labels)

            if torch.isnan(loss_gen):
                loss_gen = torch.tensor(0.0, device=DEVICE,
requires_grad=True)
            else:
                loss_gen = loss_gen.float()

            loss = loss_type + (0.1 * loss_gen)

```

```

        loss = loss / GRAD_ACCUM

        # Scaler handles backward pass safely
        scaler.scale(loss).backward()

        if (batch_idx + 1) % GRAD_ACCUM == 0:
            # Unscale gradients
            scaler.unscale_(optimizer_full)

            # Clip gradients to prevent explosions
            torch.nn.utils.clip_grad_norm_(model_a.parameters(),
MAX_GRAD_NORM)

            # Step optimizer and update scaler
            scaler.step(optimizer_full)
            scaler.update()
            optimizer_full.zero_grad()

            running_loss += loss.item() * GRAD_ACCUM
            running_type_loss += loss_type.item()
            running_gen_loss += loss_gen.item() if not
torch.isnan(loss_gen) else 0.0

            pbar.set_postfix({
                'loss': f'{running_loss/(batch_idx+1):.4f}',
                'type_l': f'{running_type_loss/(batch_idx+1):.4f}',
                'gen_l': f'{running_gen_loss/(batch_idx+1):.4f}'
            })

            val_type_acc, avg_val_ans_score = run_validation(model_a,
val_loader)
            print(f"--- Epoch {epoch+1} Summary ---")
            print(f"Val Type Acc: {val_type_acc:.4f} | REAL Val ANS Score:
{avg_val_ans_score:.4f}")

            if val_type_acc > best_val_type_acc:
                best_val_type_acc = val_type_acc
                torch.save(model_a.state_dict(), MODEL_A_FULL_STORE_PATH)
                print("Full Model checkpoint saved!")
            else:
                print("No improvement")

            print("\nPhase 4.2 Training Complete!")

```

Full Model checkpoint already exists. Skipping Phase 4.2b Training.

Evaluation 2

```
import os
import torch
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    classification_report, ConfusionMatrixDisplay,
    precision_score, recall_score, f1_score
)
from tqdm.auto import tqdm
from collections import defaultdict

# Ensure model_a and DEVICE are defined in your environment before
# running
model = model_a
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# -----
# CORE EVALUATION FUNCTION
# -----
def run_full_evaluation(model, loader, dataset_name, base_save_dir):
    """
    Runs evaluation using the exact architectural logic of
    run_validation_1.
    """
    save_path = os.path.join(base_save_dir, dataset_name)
    os.makedirs(save_path, exist_ok=True)

    print(f"\n Starting Evaluation: {dataset_name.upper()}")
    model.eval()

    all_preds, all_labels = [], []
    target_names = ['yes/no', 'number', 'other', 'unanswerable']

    # VQA-style scoring accumulators (Matching run_validation_1 logic)
    vqa_ans_score_total = 0.0
    vqa_total = 0
    per_class_vqa_score = defaultdict(float)
    per_class_count = defaultdict(int)

    with torch.no_grad():
        # tqdm wrapping the loader
        for batch in tqdm(loader, desc=f"Evaluating {dataset_name}",
            leave=True):
            pixel_values = batch['pixel_values'].to(DEVICE)
            input_ids = batch['input_ids'].to(DEVICE)
            attention_mask = batch['attention_mask'].to(DEVICE)
            # Support both common key names
```

```

        type_labels = batch.get('type_labels',
batch.get('answer_type_label')).to(DEVICE)

        with torch.cuda.amp.autocast():
            # --- MANUAL FEATURE EXTRACTION (Same as
run_validation_1) ---
            vision_outputs =
model.vision_model(pixel_values=pixel_values)
            image_features = vision_outputs.last_hidden_state[:,
1:, :]

            text_outputs = model.text_model(input_ids=input_ids,
attention_mask=attention_mask)
            text_features = text_outputs.last_hidden_state

            # --- FUSION & CLASSIFICATION ---
            fused_features = model.fusion(text_features,
image_features)
            cls_representation = fused_features[:, 0, :]
            type_logits = model.classifier(cls_representation)

            preds = torch.argmax(type_logits, dim=1)
            preds_np = preds.cpu().numpy()
            labels_np = type_labels.cpu().numpy()

            all_preds.extend(preds_np)
            all_labels.extend(labels_np)

            # VQA-style score: Correct Type = 1 match. Score = min(1,
matches/3)
            # In type-only mode, this effectively becomes accuracy *
(1/3)
            for p, l in zip(preds_np, labels_np):
                vqa_total += 1
                per_class_count[int(l)] += 1
                if p == l:
                    score = 1/3
                    vqa_ans_score_total += score
                    per_class_vqa_score[int(l)] += score

            # -----
            # METRICS CALCULATION
            # -----
            avg_vqa_ans_score = vqa_ans_score_total / vqa_total if vqa_total >
0 else 0.0

            # 1. Get dictionary for saving to CSV
            report_dict = classification_report(
                all_labels, all_preds, target_names=target_names,
output_dict=True, zero_division=0

```

```

)

# 2. PRINT THE REPORT TO CONSOLE (This is what was missing!)
print("\n" + "="*60)
print(f" CLASSIFICATION REPORT: {dataset_name.upper()}")
print("="*60)
print(classification_report(all_labels, all_preds,
target_names=target_names, zero_division=0))
print(f" VQA Score (Mean): {avg_vqa_ans_score:.4f}")
print("="*60 + "\n")

# Per-Class VQA Score Calculation
per_class_vqa = {name: (per_class_vqa_score[i] /
per_class_count[i] if per_class_count[i] > 0 else 0.0)
                  for i, name in enumerate(target_names)}

# -----
# SAVE & EXPORT
# -----
# 1. Classification Report CSV
df_report = pd.DataFrame(report_dict).transpose()
df_report.to_csv(f"{save_path}/classification_report.csv")

# 2. Summary Metrics CSV
summary = pd.DataFrame({
    "Metric": ["Accuracy", "VQA Score (Mean)", "Macro F1",
"Weighted F1"],
    "Value": [report_dict["accuracy"], avg_vqa_ans_score,
report_dict["macro avg"]["f1-score"], report_dict["weighted avg"]["f1-
score"]]
})
summary.to_csv(f"{save_path}/summary_metrics.csv", index=False)

# 3. Visual Dashboard
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# F1 Bar Chart
axes[0, 0].bar(target_names, [report_dict[n]['f1-score'] for n in
target_names], color='#C44E52')
axes[0, 0].set_title(f"F1-score per Class ({dataset_name})")
axes[0, 0].set_ylim(0, 1)

# VQA Score Bar Chart
axes[0, 1].bar(target_names, [per_class_vqa[n] for n in
target_names], color='#DD8452')
axes[0, 1].set_title("VQA Score per Class")
axes[0, 1].set_ylim(0, 0.4) # Max possible is 0.33

# Support (Sample Distribution)
axes[1, 0].bar(target_names, [report_dict[n]['support'] for n in

```

```

target_names], color='#8172B2')
    axes[1, 0].set_title("Support per Class")

    # Confusion Matrix
    ConfusionMatrixDisplay.from_predictions(all_labels, all_preds,
display_labels=target_names, cmap="Blues", ax=axes[1, 1])
    axes[1, 1].set_title("Confusion Matrix")

    plt.tight_layout()
    plt.savefig(f"{save_path}/evaluation_dashboard.png")
    plt.close()

    print(f"--- {dataset_name.capitalize()} Evaluation Complete ---")
    return report_dict["accuracy"]

# -----
# MAIN EXECUTION BLOCK
# -----
BASE_SAVE_DIR = "evaluations/model_a_ft"

# 1. Run on Val
val_accuracy = run_full_evaluation(model, val_loader, "validation",
BASE_SAVE_DIR)

# 2. Run on Test
test_accuracy = run_full_evaluation(model, test_loader, "test",
BASE_SAVE_DIR)

print(f"\nFinal Comparison:")
print(f"Validation Accuracy: {val_accuracy:.4f}")
print(f"Test Accuracy:      {test_accuracy:.4f}")

Starting Evaluation: VALIDATION

{"model_id": "e2042fc8b92143c585d6d8c50ef8cd8a", "version_major": 2, "version_minor": 0}

```

=====				
CLASSIFICATION REPORT: VALIDATION				
=====				
	precision	recall	f1-score	support
yes/no	0.59	0.90	0.71	96
number	0.31	0.21	0.25	19
other	0.73	0.82	0.78	1329
unanswerable	0.55	0.39	0.46	716
accuracy			0.68	2160
macro avg	0.54	0.58	0.55	2160


```

weighted avg      0.66      0.68      0.66      2160

VQA Score (Mean): 0.2261
=====

--- Validation Evaluation Complete ---

Starting Evaluation: TEST

{"model_id": "9ceea3e6d7564ed292b0b9677b924145", "version_major": 2, "version_minor": 0}

=====
CLASSIFICATION REPORT: TEST
=====

```

	precision	recall	f1-score	support
yes/no	0.57	0.79	0.66	99
number	0.57	0.14	0.22	29
other	0.75	0.82	0.78	1362
unanswerable	0.51	0.39	0.45	669
accuracy			0.68	2159
macro avg	0.60	0.54	0.53	2159
weighted avg	0.66	0.68	0.67	2159

```

VQA Score (Mean): 0.2266
=====

--- Test Evaluation Complete ---

Final Comparison:
Validation Accuracy: 0.6782
Test Accuracy:      0.6799

```

Clean Up

```

# Clean up

import torch
import gc
import os

os.environ["PYTORCH_ALLOC_CONF"] = "expandable_segments:True"

# 1. Explicitly delete all large objects associated with Model A
# Using try/except in case they were already deleted or don't exist in
# this scope
try:
    del model_a

```

```

except: pass
try:
    del optimizer_full
except: pass
try:
    del optimizer_stab
except: pass
try:
    del scheduler
except: pass
try:
    del train_loader
except: pass
try:
    del val_loader
except: pass
try:
    del test_loader
except: pass
try:
    del full_dataset_hi
except: pass

# 2. Force Python's garbage collector to immediately reclaim the
memory
gc.collect()

# 3. Force PyTorch to release the cached memory back to the GPU
torch.cuda.empty_cache()

# 4. Verify the memory is actually free
if torch.cuda.is_available():
    alloc = torch.cuda.memory_allocated() / (1024**3)
    reserved = torch.cuda.memory_reserved() / (1024**3)
    print(f"GPU VRAM Status after clearing:")
    print(f"  Allocated: {alloc:.2f} GB")
    print(f"  Reserved:  {reserved:.2f} GB")

GPU VRAM Status after clearing:
Allocated: 1.59 GB
Reserved:  1.85 GB

```

Phase 5: Feature Extraction

```

import os
import torch
import json
from PIL import Image
from transformers import CLIPVisionModel, CLIPIImageProcessor
from tqdm.notebook import tqdm

```

```

# =====
# CONFIGURATION
# =====
TRAIN_JSON_HI = "main_dataset_hi.json"
FEATURES_SAVE_PATH = "/kaggle/working/clip_features.pt"
FEATURES_DATASET_PATH = "/kaggle/input/datasets/aadarshpp/viziwiz-
mlpi-hi/clip_features.pt"
DEVICE = "cuda:0"

# =====
# FEATURE EXTRACTION LOGIC
# =====
if not os.path.exists(FEATURES_SAVE_PATH):
    if os.path.exists(FEATURES_DATASET_PATH):
        copy_checkpoint_to_working(FEATURES_DATASET_PATH,
FEATURES_SAVE_PATH)

if os.path.exists(FEATURES_SAVE_PATH):
    print("Pre-extracted features found! Loading from disk...")
    feat_data = torch.load(FEATURES_SAVE_PATH)
    features_lookup = {fid: feat_data['features'][i] for i, fid in
enumerate(feat_data['ids'])}
    IMAGE_FEATURE_DIM = feat_data['features'].shape[1]
    print(f"Loaded features for {len(features_lookup)} images. Feature
Dim: {IMAGE_FEATURE_DIM}")
else:
    print("No saved features found. Starting CLIP extraction ...")
    feature_extractor =
CLIPImageProcessor.from_pretrained("openai/clip-vit-large-patch14")
    vision_model = CLIPVisionModel.from_pretrained("openai/clip-vit-
large-patch14", torch_dtype=torch.float16).to(DEVICE)
    vision_model.eval()

    with open(TRAIN_JSON_HI, 'r', encoding='utf-8') as f:
        data_extract = json.load(f)

    features_dict = {}
    with torch.no_grad():
        for item in tqdm(data_extract, desc="Extracting Features"):
            img_id = item['id']
            try:
                image = Image.open(item['image_path']).convert("RGB")
                pixel_values = feature_extractor(images=image,
return_tensors="pt").pixel_values.to(DEVICE, torch.float16)
                feature =
vision_model(pixel_values=pixel_values).last_hidden_state[:, 0, :] #
CLS token
                features_dict[img_id] = feature.cpu()

```

```

        except:
            features_dict[img_id] = torch.zeros((1, 1024)) #
Fallback for corrupt images

    all_ids = [item['id'] for item in data_extract]
    features_tensor = torch.cat([features_dict[id] for id in all_ids],
dim=0)
    torch.save({'ids': all_ids, 'features': features_tensor},
FEATURES_SAVE_PATH)

    # Load into memory for Phase 6
    features_lookup = {fid: features_tensor[i] for i, fid in
enumerate(all_ids)}
    IMAGE_FEATURE_DIM = features_tensor.shape[1]

    # Clear VRAM
    del vision_model, feature_extractor
    torch.cuda.empty_cache()
    print("Feature extraction complete and saved!")

print("\nPhase 5 Complete!")

Pre-extracted features found! Loading from disk...
Loaded features for 1 images. Feature Dim: 1024

Phase 5 Complete!

```

Phase 6: Training

```

import os
import shutil
import torch
import torch.nn as nn
import json
import random
from transformers import MT5ForConditionalGeneration, AutoTokenizer
from torch.utils.data import Dataset, DataLoader
from tqdm.notebook import tqdm

# =====
# CONFIGURATION
# =====
TRAIN_JSON_HI = "main_dataset_hi.json"

# Save paths
MODEL_SAVE_ACC_PATH = "/kaggle/working/model_best_accuracy.pt"
MODEL_SAVE_BAL_PATH = "/kaggle/working/model_best_balanced.pt"

# Search directories for existing checkpoints
SEARCH_PATHS = [

```

```

    "/kaggle/working/",
    "/kaggle/input/datasets/aadarshpp/viziwiz-mlpi-hi/"
]

DEVICE = "cuda:0"
BATCH_SIZE = 48
EPOCHS = 6
LEARNING_RATE = 5e-5
IMAGE_FEATURE_DIM = 1024

# --- AGGRESSIVE Class-balanced loss knobs ---
USE_CLASS_WEIGHTS = True
WEIGHT_POWER = 1.0
BETA = 1.0
BALANCE_WARMUP_STEPS = 500

# =====
# STEP 1: Data Preparation
# =====
with open(TRAIN_JSON_HI, 'r', encoding='utf-8') as f:
    raw_data = json.load(f)

train_list = [x for x in raw_data if x['split'] == 'train']
val_list = [x for x in raw_data if x['split'] == 'val']

# =====
# CLASS DISTRIBUTION
# =====
class_counts = {}
for item in train_list:
    atype = item['answer_type']
    class_counts[atype] = class_counts.get(atype, 0) + 1

total_samples = len(train_list)
n_classes = len(class_counts)

print(f"Train: {total_samples} total samples")
for atype, count in sorted(class_counts.items()):
    print(f" {atype:15s}: {count:5d} ({count/total_samples*100:.1f}%)" )

# =====
# CLASS WEIGHTS
# =====
if USE_CLASS_WEIGHTS:
    raw_weights = {}
    for atype, count in class_counts.items():
        raw_weights[atype] = (total_samples / (n_classes * count)) **
WEIGHT_POWER

```

```

    avg_weight = sum(raw_weights[atype] * count for atype, count in
class_counts.items()) / total_samples
    class_weights = {k: v / avg_weight for k, v in
raw_weights.items()}

    print(f"\n--- Class Weights ---")
    for atype in sorted(class_weights):
        print(f" {atype:15s}: {class_weights[atype]:.3f}")
else:
    class_weights = {atype: 1.0 for atype in class_counts}

ATYPE_TO_IDX = {'yes/no': 0, 'number': 1, 'other': 2, 'unanswerable':
3}

# =====
# DATASET
# =====
class FastVQADataset(Dataset):
    def __init__(self, data_list, features_lookup, mt5_tokenizer,
lang='hi'):
        self.data = data_list
        self.feats_lookup = features_lookup
        self.mt5_tok = mt5_tokenizer
        self.lang = lang

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]
        text = item.get('question_hi', item['question']) if self.lang
== 'hi' else item['question']
        ans_text = item['answers_hi'][0]['answer'] if self.lang ==
'hi' else item['answers'][0]['answer']
        input_text = f"Question: {text} Answer:"

        source_inputs = self.mt5_tok(input_text, padding='max_length',
truncation=True, max_length=40, return_tensors="pt")
        target_inputs = self.mt5_tok(ans_text, padding='max_length',
truncation=True, max_length=10, return_tensors="pt")

        labels = target_inputs.input_ids.squeeze(0)
        labels[labels == self.mt5_tok.pad_token_id] = -100

        img_feat = self.feats_lookup.get(item['id'],
torch.zeros(IMAGE_FEATURE_DIM))

        return {
            'input_ids': source_inputs.input_ids.squeeze(0),
            'attention_mask': source_inputs.attention_mask.squeeze(0),

```

```

        'img_feat': img_feat,
        'labels': labels,
        'answer_type_idx': ATYPE_TO_IDX.get(item['answer_type'],
2),
        'orig_idx': idx
    }

# =====
# TOKENIZER + DATALOADER
# =====
print("\nInitializing tokenizer...")
mt5_tokenizer = AutoTokenizer.from_pretrained("google/mt5-small")

train_dataset = FastVQADataset(train_list, features_lookup,
mt5_tokenizer, lang='hi')
val_dataset = FastVQADataset(val_list, features_lookup, mt5_tokenizer,
lang='hi')

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=True)

# =====
# GPU CLASS WEIGHT TENSOR
# =====
weight_tensor = torch.zeros(len(ATYPE_TO_IDX), device=DEVICE)
for atype, idx in ATYPE_TO_IDX.items():
    weight_tensor[idx] = class_weights.get(atype, 1.0)

print(f"GPU weight tensor: {weight_tensor.tolist()}")

# =====
# MODEL
# =====
class mT5_VQA_Bridge(nn.Module):
    def __init__(self, img_dim=1024):
        super().__init__()
        self.mt5 =
MT5ForConditionalGeneration.from_pretrained("google/mt5-small")
        d_model = self.mt5.config.d_model
        self.img_proj = nn.Linear(img_dim, d_model)

    def forward(self, input_ids, attention_mask, img_feat,
labels=None):
        inputs_embeds = self.mt5.get_input_embeddings()(input_ids)
        img_feat = img_feat.float()
        img_embed = self.img_proj(img_feat).unsqueeze(1)
        inputs_embeds = torch.cat([img_embed, inputs_embeds], dim=1)

```

```

        img_mask = torch.ones(img_embed.shape[:2], dtype=torch.long,
device=attention_mask.device)
        attention_mask = torch.cat([img_mask, attention_mask], dim=1)

        outputs = self.mt5(inputs_embeds=inputs_embeds,
attention_mask=attention_mask, labels=labels)
        return outputs.loss, outputs.logits

    def generate(self, input_ids, attention_mask, img_feat):
        inputs_embeds = self.mt5.get_input_embeddings()(input_ids)
        img_feat = img_feat.float()
        img_embed = self.img_proj(img_feat).unsqueeze(1)
        inputs_embeds = torch.cat([img_embed, inputs_embeds], dim=1)

        img_mask = torch.ones(img_embed.shape[:2], dtype=torch.long,
device=attention_mask.device)
        attention_mask = torch.cat([img_mask, attention_mask], dim=1)

        gen_ids = self.mt5.generate(inputs_embeds=inputs_embeds,
attention_mask=attention_mask, max_new_tokens=10, do_sample=False)
        return gen_ids

# =====
# CLASS BALANCED LOSS
# =====
def compute_class_balanced_loss(logits, labels, answer_type_idx,
weight_tensor, beta=1.0):
    loss_fct = nn.CrossEntropyLoss(reduction='none', ignore_index=-
100)

    # T5 alignment: logits[t] directly predicts labels[t]. NO SHIFTING
needed.
    per_token_loss = loss_fct(logits.view(-1, logits.size(-1)),
labels.view(-1))
    per_token_loss = per_token_loss.view(labels.size())

    non_ignore = (labels != -100).float()
    per_sample_loss = (per_token_loss * non_ignore).sum(dim=-1) /
non_ignore.sum(dim=-1).clamp(min=1)

    # Weighted loss
    sample_weights = weight_tensor[answer_type_idx]
    weighted_loss = (per_sample_loss * sample_weights).mean()

    # Balance penalty
    balance_penalty = torch.tensor(0.0, device=logits.device)
    if beta > 0:
        unique_classes = answer_type_idx.unique()
        if len(unique_classes) > 1:
            class_avgs = []

```



```

        for c in unique_classes:
            mask = (answer_type_idx == c)
            if mask.sum() > 0:
                class_avgs.append(per_sample_loss[mask].mean())

        if len(class_avgs) > 1:
            cat = torch.stack(class_avgs)
            balance_penalty = cat.std() / cat.mean().clamp(min=1e-
6)

        total_loss = weighted_loss + beta * balance_penalty
        return total_loss, weighted_loss.detach(),
balance_penalty.detach()

# =====
# VALIDATION
# =====
def run_validation(model, loader, dataset_ref):
    model.eval()
    val_ans_score_total = 0.0
    val_total = 0

    type_scores = {'yes/no': [], 'number': [], 'other': [],
'unanswerable': []}
    prediction_counts = {}

    with torch.no_grad():
        for batch in tqdm(loader, desc="[Val]", leave=False):
            input_ids = batch['input_ids'].to(DEVICE)
            attn_mask = batch['attention_mask'].to(DEVICE)
            img_feats = batch['img_feat'].to(DEVICE)
            orig_idx = batch['orig_idx']

            generated_ids = model.generate(input_ids, attn_mask,
img_feats)
            pred_ans_texts = mt5_tokenizer.batch_decode(generated_ids,
skip_special_tokens=True)

            for pred_text, idx in zip(pred_ans_texts, orig_idx):
                # Convert tensor idx to int for list indexing safety
                idx_int = idx.item() if torch.is_tensor(idx) else idx

                gt_answers = [
                    dataset_ref.data[idx_int]['answers_hi'][j]
                    for j in range(min(10,
len(dataset_ref.data[idx_int]['answers_hi'])))
                ]
                ans_type = dataset_ref.data[idx_int]['answer_type']

```

```

        matches = sum(1 for gt in gt_answers if
pred_text.strip().lower() == gt.strip().lower())
        score = min(1.0, matches / 3.0)

        val_ans_score_total += score
        val_total += 1
        type_scores[ans_type].append(score)

        pred_clean = pred_text.strip().lower()
        prediction_counts[pred_clean] =
prediction_counts.get(pred_clean, 0) + 1

    overall_score = val_ans_score_total / val_total if val_total > 0
else 0

    class_avgs = {}
    for atype, scores in type_scores.items():
        class_avgs[atype] = sum(scores) / len(scores) if scores else
0.0

    # --- Balance Metric: Standard Deviation of Class Accuracies ---
    # Lower std = all classes have similar accuracy (more balanced)
    accs = [class_avgs.get('yes/no', 0.0), class_avgs.get('number',
0.0), class_avgs.get('other', 0.0), class_avgs.get('unanswerable',
0.0)]
    balance_std = torch.tensor(accs).std().item()
    acc_spread = max(accs) - min(accs)

    print("\n--- Class-wise VizWiz ANS Score ---")
    for atype, scores in type_scores.items():
        if scores:
            print(f" {atype:15s}: {sum(scores)/len(scores):.4f}")

    print(f"\nOverall Score:      {overall_score:.4f}")
    print(f"Accuracy Spread:      {acc_spread:.4f} (lower = more
balanced)")
    print(f"Accuracy Std:      {balance_std:.4f} (lower = more
balanced)")

    return overall_score, balance_std

# =====
# INITIALIZE MODEL
# =====
print("\nInitializing mT5 VQA Model...")
model_mt5 = mT5_VQA_Bridge().to(DEVICE)

found_checkpoint = False
BALANCED_FILE = "model_best_balanced.pt"
ACCURACY_FILE = "model_best_accuracy.pt"

```

```

# =====
# CHECK FOR BALANCED CHECKPOINT
# =====
for base_path in SEARCH_PATHS:
    bal_path = os.path.join(base_path, BALANCED_FILE)
    if os.path.exists(bal_path):
        print(f"Found balanced checkpoint: {bal_path}")
        model_mt5.load_state_dict(torch.load(bal_path,
map_location=DEVICE))
        found_checkpoint = True
        break

# =====
# CHECK FOR ACCURACY CHECKPOINT
# =====
if not found_checkpoint:
    for base_path in SEARCH_PATHS:
        acc_path = os.path.join(base_path, ACCURACY_FILE)
        if os.path.exists(acc_path):
            print(f"Found accuracy checkpoint: {acc_path}")
            model_mt5.load_state_dict(torch.load(acc_path,
map_location=DEVICE))
            found_checkpoint = True
            break

# =====
# RUN VALIDATION IF CHECKPOINT EXISTS
# =====
if found_checkpoint:
    print("\nCheckpoint loaded. Skipping training.")
    val_score, val_std = run_validation(model_mt5, val_loader,
val_dataset)
    print(f"\nOverall Val Score: {val_score:.4f} ({val_score*100:.2f}
%) | Balance Std: {val_std:.4f}")

# =====
# TRAINING
# =====
else:
    print("\nNo checkpoint found. Starting AGGRESSIVE class-balanced
training...")
    optimizer = torch.optim.AdamW(model_mt5.parameters(),
lr=LEARNING_RATE, weight_decay=0.01)

    best_accuracy = 0.0
    # For balanced model: we want to MINIMIZE standard deviation
    best_balanced_std = float('inf')
    best_balanced_acc = 0.0

```

```

global_step = 0

for epoch in range(EPOCHS):
    model_mt5.train()
    epoch_loss = 0
    epoch_wt = 0
    epoch_bal = 0

    pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}")
    for batch in pbar:
        optimizer.zero_grad()

        input_ids = batch['input_ids'].to(DEVICE)
        attn_mask = batch['attention_mask'].to(DEVICE)
        img_feats = batch['img_feat'].to(DEVICE)
        labels = batch['labels'].to(DEVICE)
        atype_idx = batch['answer_type_idx'].to(DEVICE)

        _, logits = model_mt5(input_ids, attn_mask, img_feats,
labels)

        effective_beta = BETA * min(1.0, global_step /
max(BALANCE_WARMUP_STEPS, 1))

        total_loss, wt_loss, bal_penalty =
compute_class_balanced_loss(
            logits, labels, atype_idx, weight_tensor,
beta=effective_beta
        )

        total_loss.backward()
        torch.nn.utils.clip_grad_norm_(model_mt5.parameters(),
1.0)

        optimizer.step()

        epoch_loss += total_loss.item()
        epoch_wt += wt_loss.item()
        epoch_bal += bal_penalty.item()
        global_step += 1

    pbar.set_postfix({
        'loss': f'{total_loss.item():.3f}',
        'wt': f'{wt_loss.item():.3f}',
        'bal': f'{bal_penalty.item():.3f}',
        'β': f'{effective_beta:.2f}'
    })

    n_batches = len(train_loader)
    avg_loss = epoch_loss / n_batches
    avg_wt = epoch_wt / n_batches

```

```

    avg_bal = epoch_bal / n_batches

    val_score, val_std = run_validation(model_mt5, val_loader,
val_dataset)

    print(f"\nEpoch {epoch+1} Summary:")
    print(f"  Loss: total={avg_loss:.4f} | weighted={avg_wt:.4f} |
balance_penalty={avg_bal:.4f}")
    print(f"  Val: overall={val_score:.4f} ({val_score*100:.2f}%)
| balance_std={val_std:.4f}")

    # 1. Save best accuracy model
    if val_score > best_accuracy:
        best_accuracy = val_score
        torch.save(model_mt5.state_dict(), MODEL_SAVE_ACC_PATH)
        print(f"  Saved BEST ACCURACY model
({best_accuracy:.4f})")

    # 2. Save best balanced model (Lowest Std = most balanced)
    # Only consider models with >15% accuracy to avoid saving a
perfectly balanced 0% model
    if val_score > 0.15:
        is_better_balance = False

    # If standard deviation is strictly lower, it's more
balanced
    if val_std < best_balanced_std - 0.005:
        is_better_balance = True
    # If standard deviation is roughly the same (within 0.005
margin), tie-break with accuracy
    elif abs(val_std - best_balanced_std) <= 0.005:
        if val_score > best_balanced_acc:
            is_better_balance = True

    if is_better_balance:
        best_balanced_std = val_std
        best_balanced_acc = val_score
        torch.save(model_mt5.state_dict(),
MODEL_SAVE_BAL_PATH)
        print(f"  Saved BEST BALANCED model (Std:
{best_balanced_std:.4f}, Acc: {best_balanced_acc:.4f})")

    print("\nTraining Complete!")
    print(f"Best Accuracy Score: {best_accuracy:.4f}")
    print(f"Best Balanced Score: Std={best_balanced_std:.4f} (with
Acc={best_balanced_acc:.4f})")

Train: 20523 total samples
number      :   301 (1.5%)
other       : 13733 (66.9%)

```

```
unanswerable : 5532 (27.0%)
yes/no       : 957 (4.7%)
```

--- Class Weights ---

```
number       : 17.046
other        : 0.374
unanswerable : 0.927
yes/no       : 5.361
```

Initializing tokenizer...

GPU weight tensor: [5.361285209655762, 17.04568099975586, 0.3736073672771454, 0.9274674654006958]

Initializing mT5 VQA Model...

```
{"model_id": "f8cfbe1de58c4a15b4ea0fea38f6a85a", "version_major": 2, "version_minor": 0}
```

Found balanced checkpoint: /kaggle/input/datasets/aadarshpp/viziwiz-mlpi-hi/model_best_balanced.pt

Checkpoint loaded. Skipping training.

```
{"model_id": "", "version_major": 2, "version_minor": 0}
```

--- Class-wise VizWiz ANS Score ---

```
yes/no       : 0.5972
number       : 0.0877
other        : 0.2034
unanswerable : 0.5591
```

```
Overall Score:      0.3378
Accuracy Spread:    0.5095 (lower = more balanced)
Accuracy Std:       0.2547 (lower = more balanced)
```

Overall Val Score: 0.3378 (33.78%) | Balance Std: 0.2547

Evaluation 3

```
# E3
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch.utils.data import DataLoader
from tqdm import tqdm

# =====
```

```

# SAVE DIR
# =====
SAVE_DIR_MT5 = "evaluations/model_b_mt5"
os.makedirs(SAVE_DIR_MT5, exist_ok=True)

print("\n" + "="*60)
print("EVALUATING MODEL B (4.3 mT5 BALANCED)")
print("="*60)

# =====
# DATA LOADING
# =====
TRAIN_JSON_HI = "main_dataset_hi.json"

with open(TRAIN_JSON_HI, 'r', encoding='utf-8') as f:
    raw_data = json.load(f)

test_list_mt5 = [x for x in raw_data if x['split'] == 'test']

test_dataset_mt5 = FastVQADataset(
    test_list_mt5,
    features_lookup,
    mt5_tokenizer,
    lang='hi'
)

test_loader_mt5 = DataLoader(
    test_dataset_mt5,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

# =====
# EVALUATION LOOP (Pure VizWiz ANS Score Logic)
# =====
test_ans_score_total = 0.0
test_total = 0

test_ans_scores_mt5 = {'yes/no': [], 'number': [], 'other': [],
    'unanswerable': []}
prediction_counts_mt5 = {}

model_mt5.eval()

with torch.no_grad():
    for batch in tqdm(test_loader_mt5, desc="[Test Model B mT5]"):
        input_ids = batch['input_ids'].to(DEVICE)
        attn_mask = batch['attention_mask'].to(DEVICE)

```

```

img_feats = batch['img_feat'].to(DEVICE)
orig_idx = batch['orig_idx']

generated_ids = model_mt5.generate(input_ids, attn_mask,
img_feats)
pred_texts = mt5_tokenizer.batch_decode(generated_ids,
skip_special_tokens=True)

for pred_text, idx in zip(pred_texts, orig_idx):
    idx_int = idx.item() if torch.is_tensor(idx) else idx
    item = test_dataset_mt5.data[idx_int]
    ans_type = item['answer_type']

    # VizWiz ANS Score calculation
    gt_answers = [item['answers_hi'][j]['answer'] for j in
range(min(10, len(item['answers_hi'])))]
    matches = sum(1 for gt in gt_answers if
pred_text.strip().lower() == gt.strip().lower())
    score = min(1.0, matches / 3.0)

    test_ans_score_total += score
    test_total += 1
    test_ans_scores_mt5[ans_type].append(score)

    pred_clean = pred_text.strip().lower()
    prediction_counts_mt5[pred_clean] =
prediction_counts_mt5.get(pred_clean, 0) + 1

# =====
# METRICS CALCULATION
# =====
overall_ans_mt5 = test_ans_score_total / test_total if test_total > 0
else 0.0
class_ans_mt5 = {k: np.mean(v) if len(v) > 0 else 0.0 for k, v in
test_ans_scores_mt5.items()}

# Balance Metrics: Standard Deviation and Spread
accs = [class_ans_mt5.get('yes/no', 0.0), class_ans_mt5.get('number',
0.0), class_ans_mt5.get('other', 0.0),
class_ans_mt5.get('unanswerable', 0.0)]
balance_std = torch.tensor(accs).std().item()
acc_spread = max(accs) - min(accs)

# =====
# SAVE METRICS
# =====
# Summary Table
summary_mt5 = pd.DataFrame({
    "Metric": [
        "Overall ANS Score",

```



```

        "Yes/No ANS", "Number ANS", "Other ANS", "Unanswerable ANS",
        "Accuracy Spread", "Accuracy Std"
    ],
    "Value": [
        overall_ans_mt5,
        class_ans_mt5['yes/no'], class_ans_mt5['number'],
        class_ans_mt5['other'], class_ans_mt5['unanswerable'],
        acc_spread, balance_std
    ]
})
summary_mt5.to_csv(f"{SAVE_DIR_MT5}/summary_metrics.csv", index=False)

# Class-wise scores
pd.DataFrame.from_dict(class_ans_mt5, orient="index",
    columns=["ans_score"]).to_csv(
    f"{SAVE_DIR_MT5}/per_class_ans_scores.csv"
)

# Prediction frequencies
pd.DataFrame(
    sorted(prediction_counts_mt5.items(), key=lambda x: x[1],
    reverse=True),
    columns=["prediction", "count"]
).to_csv(f"{SAVE_DIR_MT5}/prediction_frequency.csv", index=False)

# =====
# DASHBOARD FIGURE
# =====
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Class-wise ANS Score
classes = list(class_ans_mt5.keys())
scores = list(class_ans_mt5.values())
axes[0].bar(classes, scores, color=['#4C72B0', '#55A868', '#C44E52',
    '#8172B2'])
axes[0].set_title("VizWiz ANS Score per Class", fontweight='bold')
axes[0].set_ylim(0, 1)
for i, v in enumerate(scores):
    axes[0].text(i, v + 0.02, f"{v:.3f}", ha='center',
    fontweight='bold')

# Plot 2: Top Predictions
top_preds = sorted(prediction_counts_mt5.items(), key=lambda x: x[1],
    reverse=True)[:10]
if top_preds:
    labels, counts = zip(*top_preds)
    axes[1].bar(labels, counts, color='#DD8452')
    axes[1].set_title("Top 10 Model Predictions", fontweight='bold')
    axes[1].tick_params(axis='x', rotation=45)

```

```

plt.suptitle("Model B (mT5 Balanced) Evaluation Dashboard",
fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig(f"{SAVE_DIR_MT5}/evaluation_dashboard.png", dpi=300)
plt.show()

# =====
# PRINT FINAL
# =====
print("\n--- MODEL B VizWiz ANS RESULTS ---")
print(f"Overall ANS Score: {overall_ans_mt5:.4f}")
for k, v in class_ans_mt5.items():
    print(f" {k:15s}: {v:.4f}")
print(f"Accuracy Spread: {acc_spread:.4f} (lower = more balanced)")
print(f"Accuracy Std: {balance_std:.4f} (lower = more
balanced)")

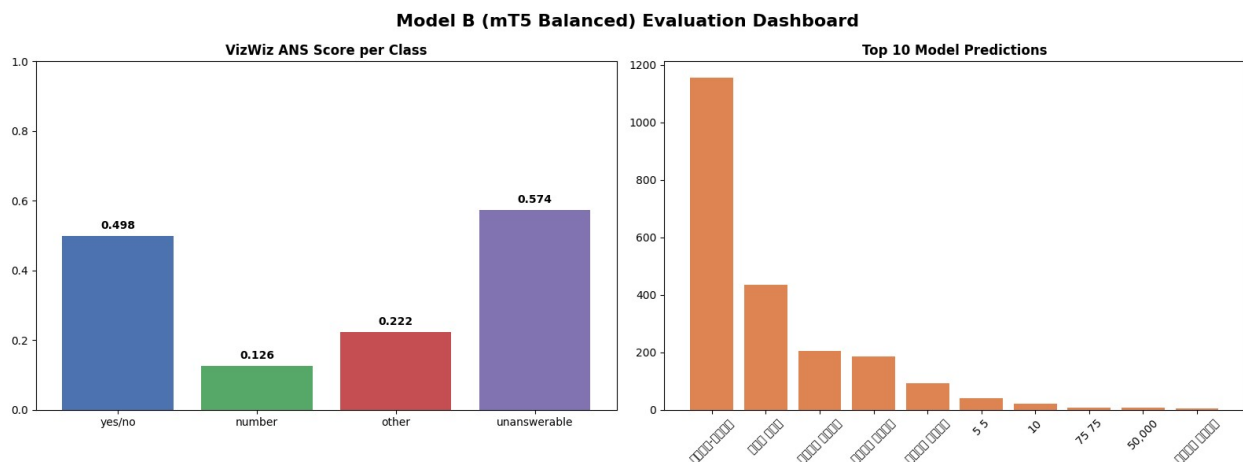
print(f"\nSaved to: {SAVE_DIR_MT5}")

```

```

=====
EVALUATING MODEL B (4.3 mT5 BALANCED)
=====

```



```

--- MODEL B VizWiz ANS RESULTS ---
Overall ANS Score: 0.3429
yes/no           : 0.4983
number          : 0.1264
other           : 0.2225
unanswerable    : 0.5745
Accuracy Spread: 0.4481 (lower = more balanced)
Accuracy Std:   0.2149 (lower = more balanced)

Saved to: evaluations/model_b_mt5

```

